Department of Mechanical Engineering

University of Victoria

Final Design Report

MECH 458 - Automatic Sorting System

Prepared By:

B01 – Group 4

Jackson Hamel: V00983960

Jayden Morgan: V00958908

Abstract

This report outlines the design, development, and performance of an automatic sorting system. The goal of the project was to design a conveyor-based sorting mechanism capable of identifying and categorizing four object types (black plastic, white plastic, aluminum, and steel) based on material and color. The system uses a combination of sensors including a reflective sensor, ferromagnetic sensor, end-of-travel sensor, and Hall-effect sensor, all integrated through an ATmega2560 microcontroller.

The system operates through state and interrupt-driven logic to ensure responsive and quick processing. A stepper motor-controlled rotating plate performs the sorting, while a DC motor drives the conveyor belt. Key features include a linked list system for continuous object handling, pause and ramp-down interrupts for user control, and an efficient algorithm for minimal rotation of the sorting plate.

Performance testing focused on sorting accuracy, response time, and system reliability. Challenges included optimizing sensor placement, fine-tuning ADC readings, and implementing s-curve acceleration for smooth motor control. Results demonstrated successful sorting with high accuracy and efficiency, validating the system design and logic.

Table of Contents

Abstract 2
Introduction 7
Project Assignment 7
Apparatus 7
Project Objectives 8
Acknowledgements 8
Methods and Design Approach 10
Design Overview 10
10-Bit ADC 10
Linked List Queuing 11
Stepper Motor Control and Plate Position 12
Interrupt-Driven Control System 12
Project Timeline 13
Results and System Technical Documentation 14
High Level Breakdown 14
Circuit Diagrams 15
System Algorithm Description 16
Initialization 16
State Machine 17
Add to Queue 19
Moving Sorting Plate 20
Pause Interrupt 21
Ramp Down 22
System Performance 23
Operating Parameters 23
Testing & Calibration Procedures 25
Calibrating Pieces 25
Calibrating Piece Drop Time & Position 26
Stepper Motor Maximum Speed Testing 26
Demonstration Results 27
Limitations & Trade-offs 27
Experience & Recommendations 28

Conclusion 29
References 30
Appendix 31

List of Figures

Figure 1: Sorting System Apparatus [3] 8
Figure 2: Gantt Chart 13
Figure 3: Main Routine Block Diagram 14
Figure 4: Circuit Diagram 15
Figure 5: Board Wiring 16
Figure 6 Stepper Homing Sequence 17
Figure 7: State Machine Sequence 18
Figure 8: Add to Queue Pseudocode 19
Figure 9: Plate Sorting 20
Figure 10: Pause Algorithm 21
Figure 11: Ramp Down Interrupt Breakdown 22
Figure 12: 90 Degree Rotation Acceleration Profile 24
Figure 13: 180 Degree Rotation Acceleration Profile 24

List of Tables

Table 1: S-Curve Parameters.....23
Table 2: ADC Maximum Readings.....26
Table 3: Successful Test Trial SPI Scores.....27

Introduction

Project Assignment

The main task for the MECH 458 Final Project is to design, build, and demonstrate an automated conveyor belt sorting system that categorizes items by material and color, placing them into the correct sorting bucket. The system must automatically sort four object types:

- C_0: Black Plastic
- C_1: Steel
- C_2: Aluminum
- C_3: White Plastic

System performance will be based upon two main factors: accuracy and sorting time.

Apparatus

The sorting system apparatus, found in Figure 1, is made up of three main subsystems: the conveying subsystem, the inspection stations, and the sorting subsystem. The conveying subsystem consists of a conveyor belt driven by a DC motor. The overall speed of the conveyor belt will be controlled using a PWM signal, which will be discussed more in-depth later in this report. The sorting subsystem will drop each object into specific bins based on their properties and appearance. The sorting system uses a rotating tray actuated through a stepper motor. The inspection stations are made up of three different sensors:

- S_0: Ferromagnetic Sensor
- S_1: Reflective Sensor
- S_2: End of Travel Sensor
- HE: Hall-Effect Sensor

The ferromagnetic sensor, in combination with an optical sensor, will identify whether an object is metallic (aluminum or steel). The reflective sensor, also paired with an optical sensor, will assess the object's reflectiveness (shininess). The end-of-travel sensor will detect when an object reaches the end of the conveyor belt, signaling the system that it is ready for sorting. This will enable the system to adjust the conveyor belt speed and position the sorting tray accordingly. The hall-effect

sensor will establish the sorting bucket's "home" position to initialize the sorting process. This sensor will designate the default starting point for the sorting tray. Based on the integration of each of these sensors, the system must be able to determine the properties of each object and sort it into the correct location.

The system will be powered by a microcontroller unit (MCU), specifically the ATmega2560. The ATmega2560 data sheet is referenced in this report. The MCU will control the DC motor for the conveyor belt and the stepper motor for the sorting bucket. A detailed circuit design, including the MCU's pin connections, will be shown later in the report.

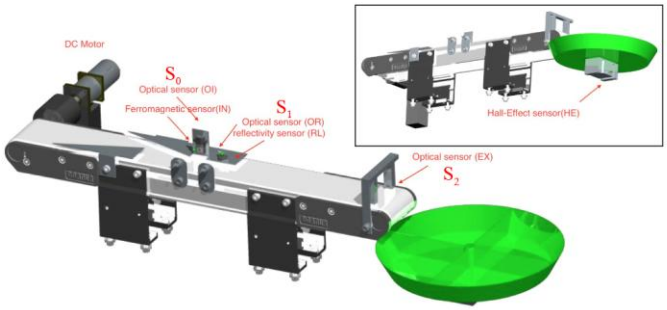


Figure 1: Sorting System Apparatus [3]

Project Objectives

The main objectives of this project include, but are not limited to [3]:

1. Sort items of varying materials and colors into the appropriate category with the greatest possible accuracy and within the shortest amount of time
2. Maximize the number of items on the conveyor belt at any given time
3. Utilize a DC motor to operate the conveyor belt
4. Utilize a stepper motor to rotate the sorting bucket, ensuring objects are grouped based on their material and visual properties
5. Integrate the use of various sensors to analyze an object's properties

Acknowledgements

Completing this project will come with several challenges, requiring planning and problem-solving. Many factors must be considered and balance ed to ensure the system operates smoothly and

efficiently. Some of the key challenges we have identified include sensor accuracy, sorting speed and efficiency, motor control and synchronization, and microcontroller integration.

Sensor accuracy is a critical factor in ensuring the sorting system is operating correctly. The sensors must be able to distinguish between different materials and colors without error, which requires precise programming and proper testing. External factors must also be considered such as lighting conditions and sensor placement. Multiple test trials will be conducted to verify that data ranges for each sensor are correctly applied and that the sensor positions are optimized for the system.

The overall sorting speed and efficiency of the system must be considered to achieve maximum accuracy. The ability to quickly sort objects while maintaining high accuracy is crucial to the success of this project. Achieving the correct balance between speed and accuracy will be challenging, as speeding up the process may risk sorting inaccuracies. To improve sorting efficiency, the movement of both the conveyor belt and sorting tray will be aimed to be optimized. This will be done through fine-tune testing of system delays and utilizing the use of an s-curve for stepper motor acceleration.

Controlling the stepper motor and ensuring its synchronization with the system will be key to the success of this project. We'll consider two main control methods: linear acceleration and s-curve acceleration. Linear acceleration is simpler to implement because it requires less complex logic than the s-curve. However, the downside is that it can cause jerky movements when the motor changes direction. On the other hand, the s-curve smooths out these sudden movements by gradually adjusting the speed during direction changes. While the s-curve is more efficient and effective, it's harder to implement due to its complexity. Our goal is to develop an s-curve through calculations and research, followed by testing to find the most efficient acceleration pattern.

Given the range of subsystems and coding routines involved in this project, successfully integrating the microcontroller to manage these systems will be crucial for its success. Coordinating multiple sensors with precise timing and accounting for system delays will be challenging and could lead to errors in the sorting process. This includes managing various inputs and outputs, like those from the sensors, while also controlling the conveyor belt and sorting bucket at the same time. To address this, we'll implement sensor debouncing, use timers for precise motor control, and focus on efficient coding. Several rounds of testing and iterations will be conducted to ensure the microcontroller integration is optimized.

Methods and Design Approach

Design Overview

The automatic sorting system is composed of three primary subsystems: the conveying system, the inspection system, and the sorting mechanism. These components work together to detect, identify, and sort objects by material and color. A DC motor drives the conveyor belt, which transports items through the inspection area to the sorting tray. Belt speed is regulated using a PWM signal, allowing for controlled and efficient motion.

Objects are analyzed as they pass through the inspection subsystem (identified using an ADC), which includes a ferromagnetic sensor to detect metal, a reflective sensor to distinguish between plastic colors, an end-of-travel sensor to signal sorting readiness, and a Hall-effect sensor to determine the sorting tray's home position. These sensors work in tandem to identify the object's material properties and generate data to be utilized for sorting accurately.

To manage multiple items simultaneously, the system utilizes a dynamic linked list queuing structure. Each object detected by the sensors is added to the queue along with its respective item code. This allows for continuous object flow and efficient sorting without waiting for one item to fully process before the next is inspected. The queue enables precise tracking of each object's properties and ensures they are sorted in the correct order.

The sorting mechanism consists of a rotating plate driven by a stepper motor, which directs each item into the appropriate bin. The ATmega2560 microcontroller coordinates all operations, processing sensor inputs, managing the queue, and executing motor control based on a state machine and interrupt-driven logic. This architecture allows for real-time response, optimized sorting speed, and reliable system performance.

10-Bit ADC

The ATmega2560 microcontroller features a 10-bit Analog-to-Digital Converter (ADC), which is used in the system to read analog values (ranging from 0 to 1023) from the reflective sensor. The ADC plays a critical role in determining the object's surface characteristics, particularly its color and

shininess, allowing the system to distinguish between black plastic, white plastic, aluminum, and steel.

To maximize accuracy, the ADC continuously samples reflectivity values as the object passes in front of the sensor. A scanning algorithm is used where the system tracks the lowest ADC value detected during the object's passage. This value is then used to classify the object, as darker or less reflective objects produce a higher ADC value compared to shinier materials. Once the ADC value begins to increase, the system assumes the object has passed the sensor area and assigns the lowest recorded value for sorting classification.

By using the full 10-bit resolution (as compared to the 8-bit resolution used in the laboratory experiments), the system ensures a fine distinction between object types, reducing the risk of errors due to overlapping ADC ranges. This issue will be discussed further in-depth later in this report, as the calibration of these ranges was essential for project success.

Linked List Queuing

The system uses a dynamic linked list structure to queue objects as they are identified by the sensors. This queue ensures that objects are sorted in the exact order they are detected, allowing multiple items to be on the conveyor belt simultaneously without identification or sorting order errors.

The queue is initialized using setup(), and new nodes are created with initLink(). As each item is scanned, its classification data is stored, and the node is added to the end of the queue using enqueue(). When the item reaches the end of the conveyor belt (exit gate sensor), dequeue() removes it from the front of the queue, ensuring a first-in-first-out behavior.

Additional functions, such as isEmpty() and size() are used to check the queue's status during pause or ramp-down states. This queuing approach enables continuous sorting and ensures proper synchronization between the inspection and sorting subsystems. The detailed code for each linked list function can be found in the Appendix.

Stepper Motor Control and Plate Position

The sorting system utilizes a stepper motor to rotate a sorting plate that directs each object into its corresponding bin. Precise control of the stepper motor is essential to ensure accurate positioning and efficient sorting time. To improve movement smoothness and minimize mechanical stress, the system implements an S-curve acceleration profile for the stepper motor. This profile gradually increases and decreases speed to prevent abrupt starts and stops (which are common in linear acceleration profiles). The S-curve profile improves reliability and allows for smoother transitions between sorting rotations.

To further improve system efficiency, the algorithm includes shortest-path rotation logic. When an object is ready to be sorted, the system first checks if the plate is already in the correct position. If movement is required, it calculates the shortest rotation distance (clockwise or counterclockwise) based on the current and target positions. This implies that only rotations of 90 or 180 degrees will occur, as rotations greater than 180 degrees are avoided to minimize unnecessary movement and reduce total sorting time.

A pre-drop mechanism is also implemented in the sorting subsystem. As the stepper motor nears the final steps of its rotation (20 steps before rotation completion), the conveyor belt resumes movement to initiate a pre-drop of the piece. This timing ensures that the object lands accurately in the intended bin, preventing it from hitting the rim of the plate and causing a sorting error.

This combination of controlled acceleration and optimized rotation logic ensures reliable and efficient sorting throughout the operation of the system. Specific system parameters of these functions will be discussed in more detail later in this report.

Interrupt-Driven Control System

The main algorithm is built around an interrupt-driven control system to enable real-time responsiveness and efficient operation. Interrupts allow the MCU to immediately respond to sudden events such as sensor detections. Using interrupts ensures that critical tasks are executed with minimal delay and that overlapping operations (such as ADC scanning while sorting) can occur without conflict.

Four primary interrupts manage the system's behavior: the pause interrupt, ramp-down interrupt, object detection interrupt, and the exit gate interrupt. These interrupts are responsible for halting or resuming the system, scanning and classifying objects, managing queue operations, and controlling the timing and movement of the sorting plate. A detailed breakdown of each interrupt and its associated logic is provided in later sections of the report.

Project Timeline

The following Gantt chart outlines the overall timeline and deadlines for specific tasks in the project. The purple section represents the design and development phase, which includes the core framework of the project, such as integrating sensors, implementing speed control for the stepper motor, and developing the sorting logic. The pink section marks the testing and optimization phase, which will begin once an initial, functional system is in place. This phase focuses on optimizing and testing the system for continuous improvement, such as enhancing code runtime, refining speed control, and preparing the project for demonstration.

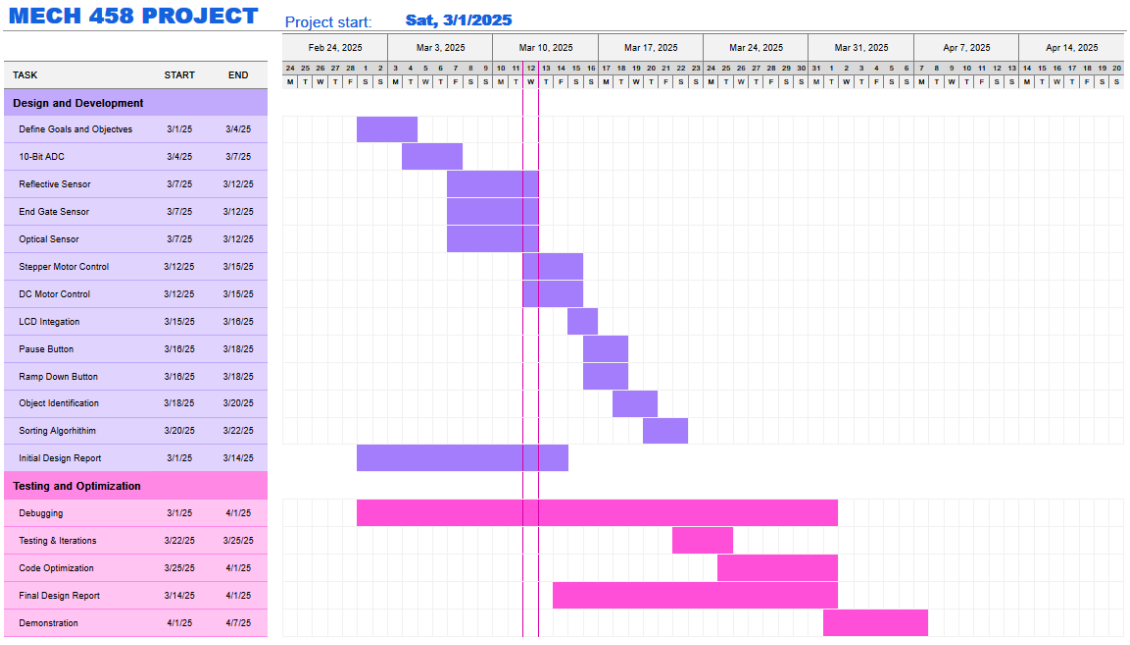


Figure 2: Gantt Chart

Results and System Technical Documentation

High Level Breakdown

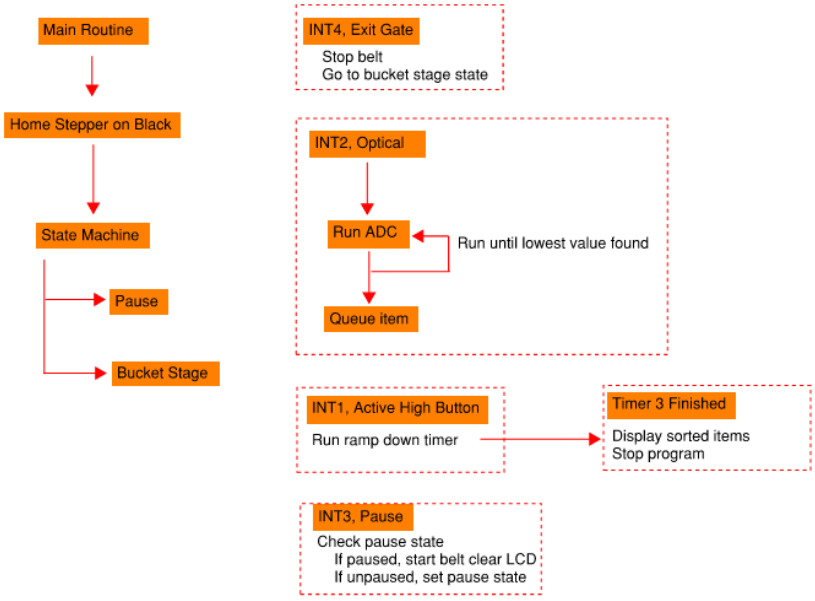


Figure 3: Main Routine Block Diagram

The main routine runs continuously, maintaining conveyor belt movement as long as items remain in the queue. To enhance system efficiency, the program first checks whether the sorting plate is already in the correct position before executing a rotation. If a 180-degree rotation is required, the system defaults to a clockwise turn. For 90-degree rotations (positive or negative), the system calculates and selects the shortest rotation direction to minimize movement time.

The main function is driven by four primary interrupts. The pause interrupt, triggered by a button press, stops the system and displays the current number of queued items and their indexes. The ramp-down interrupt initiates a ~7-second timer, allowing remaining items on the conveyor to be processed before displaying the final item count and halting the system until a full system restart occurs. In both cases, the belt motor is actively braked to VCC, ensuring a quick and controlled stop.

The object detection interrupt monitors sensor activity, analyzes the object using the reflective sensor, and adds it to the queue. The final interrupt, associated with the sorting stage, manages the plate positioning and ensures that sorting operations do not overlap. It sets a flag to prevent conflicting operations and initiates the item sorting process. Once an object reaches the exit gate,

the main routine dequeues the item, frees its allocated memory, and directs it into the appropriate bin. After the item is sorted into the bin, the system checks if the exit gate flag is set, signalling that another piece has reached the exit gate and is ready to be sorted.

Circuit Diagrams

The circuit diagram below provides a detailed visual representation of the components and their respective connections used in the project. It illustrates how each part of the system is wired together to ensure proper functionality and ease of troubleshooting.

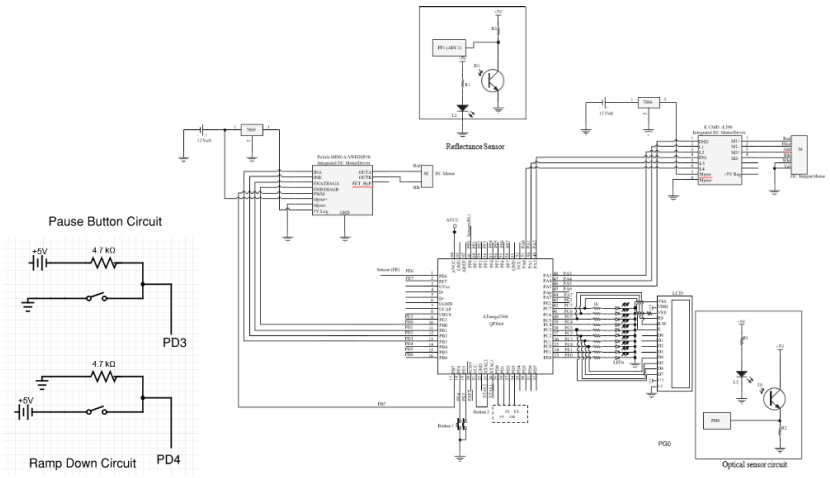


Figure 4: Circuit Diagram

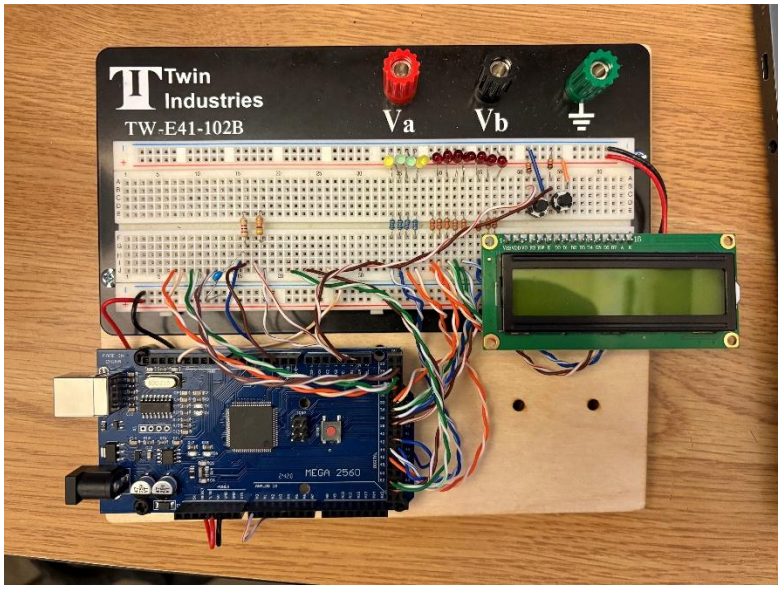


Figure 5: Board Wiring

To simplify the process, most of the wiring from the previous labs and the project will remain unchanged to reduce the time spent debugging code. If the system is wired in the same way as in the previous lab, we can refer to the lab code to determine whether the issue is related to coding logic or the mechanical setup. Simpler parts of the project, such as button inputs or sensor inputs, will be assigned to available digital or analog pins. Additionally, the breadboard is organized to streamline the setup when switching between stations. The objective is to connect the wires according to the labels on the apparatus's output.

System Algorithm Description

Initialization

Upon powering the system, all core components are energized and ready for operation. However, the conveyor belt does not immediately begin moving. Instead, the system remains idle, awaiting a manual input to start the belt. This feature was intentionally implemented to reduce unnecessary noise during lab sessions, allowing for better focus among group members while setting up or troubleshooting the system.

Before the conveyor belt is activated, the system automatically performs a homing sequence for the stepper motor, aligning the sorting plate with its default starting position. Following the lab

instructions, the stepper plate must home to the "black" bin location. This is achieved using a Hall-effect sensor positioned beneath the rotating plate. The system steps the motor through its lookup table until the sensor detects a low signal, indicating that the home position has been reached. Once homed, the stepper motor remains in position until the system begins sorting, ensuring consistent and repeatable behavior.

```
//Motor homing
int i = 0;
while((PING & 0b00000010) == 0b00000010){ // plate homing sequence
    PORTA = sTable[i];
    curPos = i; //record the last inputted position
    mTimer(15); //20ms delay for motor
    i++;
    if(i == 4) {
        i = 0;
    }
}
```

Figure 6 Stepper Homing Sequence

State Machine

The system operates using a state machine structure based on the skeleton code provided in the project folder. This framework enables smooth transitions between key system states in response to sensor inputs, all managed through interrupts. The primary states include the polling state, where the system monitors for incoming objects; the pause state, which halts system operation and displays queue information when triggered by manual input; and the bucket stage, where the stepper motor rotates the sorting plate to sort objects into the correct bins.

To ensure accurate sensor readings and prevent errors, the reflective sensor logic is handled entirely within its interrupt service routine (ISR) rather than being treated as a separate state. This approach was chosen to prevent double-triggering caused by the simultaneous activation of the reflective and end gate sensors. By processing reflectivity data immediately within the interrupt, the

system maintains reliable object identification while avoiding timing conflicts during fast-paced operation.

```
// POLLING STATE
POLLING_STAGE:
switch(STATE){
    case (0) :
    goto POLLING_STAGE;
    break;
    case (1) :
    goto PAUSE;
    break;
    case (3) :
    goto BUCKET_STAGE;
    break;
    case (5) :
    goto END;
    default :
    goto POLLING_STAGE;
}//switch STATE
```

Figure 7: State Machine Sequence

Add to Queue

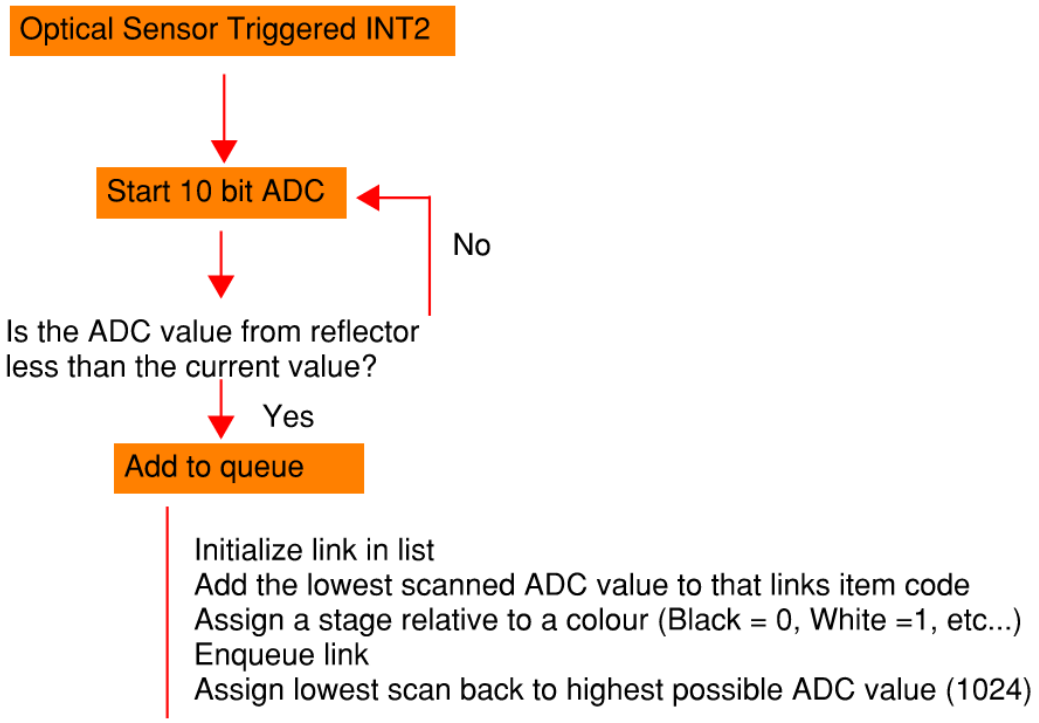


Figure 8: Add to Queue Pseudocode

Once the optical sensor is triggered, the system knows that there is an item that is to be added to the queue. Although the main problem is determining which scanned ADC value is the most accurate to correctly identify the piece. The process our group opted for included starting the ADC the moment the part breaks the sensor and continuously searching until the lowest value is found by then immediately adding the part to the queue once the next ADC value is higher than the previous. At this point, the item is initialized, the ADC value is stored in the item code, and the piece identifier is stored for future use.

The item is added to the queue within the interrupt itself as opposed to the bucket stage to allow for continuous sorting (stacking the belt). This is because when the program is inside an interrupt, the exit gate would suddenly stop the ADC from reading, causing errors in the ADC scanned value. By leaving it in the interrupt, the ADC can finish it's scan, add the item, and then continue the program.

Moving Sorting Plate

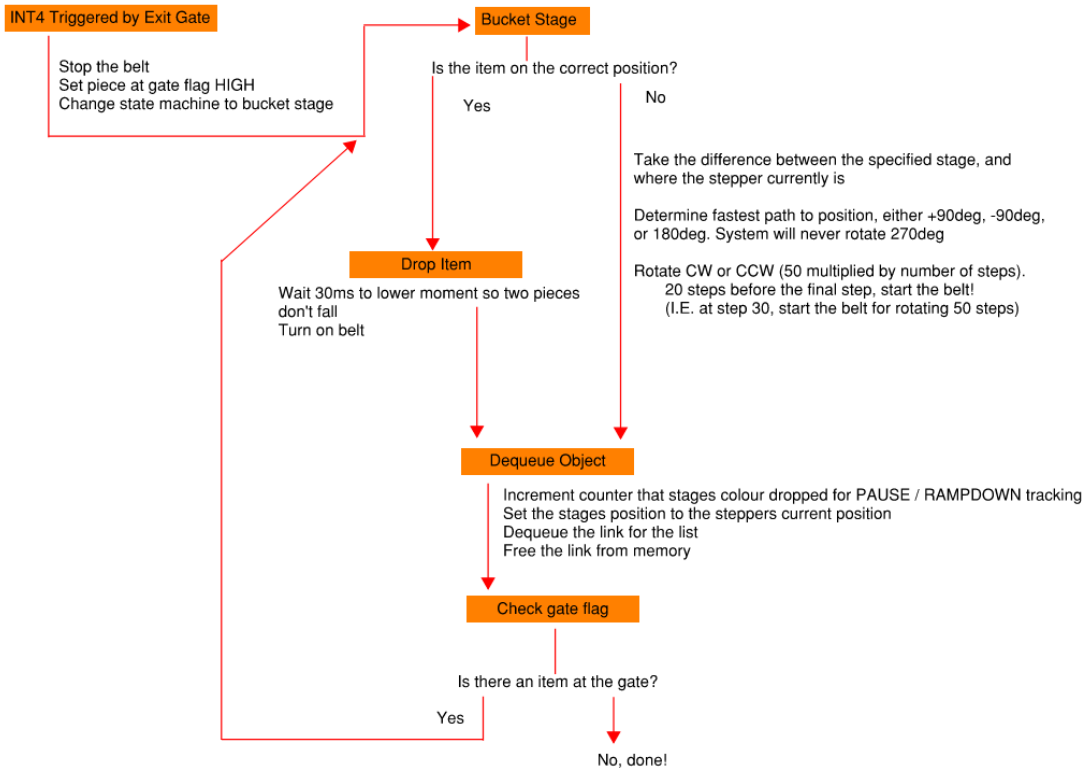


Figure 9: Plate Sorting

The plate movement scheme starts when an interrupt is triggered, typically by a sensor detecting an item. This interrupt immediately stops the belt and sets a flag to indicate that an item is present. At this point, the system checks whether the plate is already in the correct position to receive the item. If it is, the item is dropped immediately. If not, the algorithm calculates the shortest rotational distance needed to align the plate with the correct destination. This is done by subtracting the current position of the plate from the required target position, ensuring the rotation does not exceed 180 degrees to maintain efficiency.

Once the direction and number of steps are determined, the stepper motor begins rotating the plate. As the stepper motor nears the end of its rotation (specifically 20 steps before completion) the belt begins moving again. This early belt movement is a pre-drop mechanism designed to ensure that the item lands properly in the intended slot and does not hit the rim of the plate due to a delayed drop. After the item has been successfully placed, it is dequeued from the system's item

queue. The flag is then checked to see if another item is waiting. If there is, the entire process is restarted to handle the next item in line.

Pause Interrupt

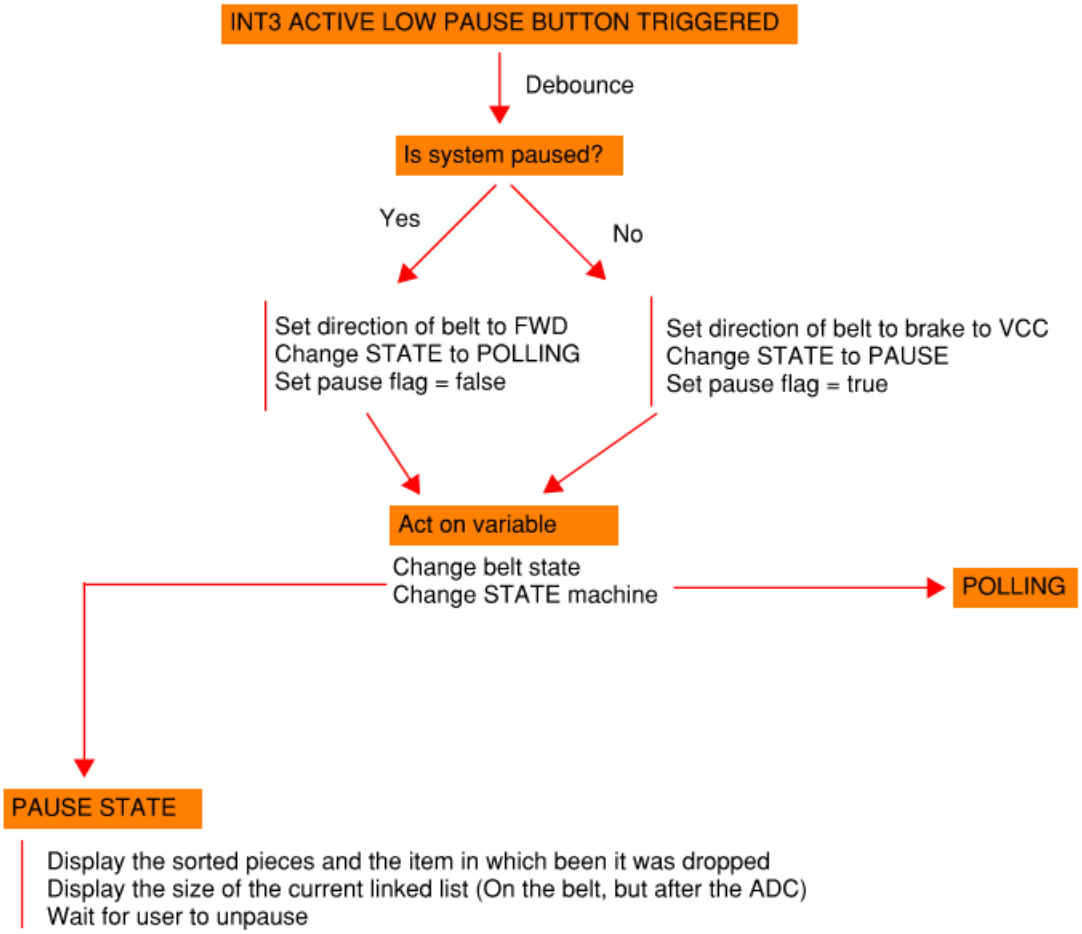


Figure 10: Pause Algorithm

The pause feature is implemented using a simple algorithm that relies on a single flag to track whether the system is currently paused. When the pause interrupt is triggered and properly debounced, the program uses a conditional check to change the system state, update the flag, and switch the belt motor on or off accordingly. Depending on the flag status, the system transitions between the polling state and the pause state. While in the pause state, the system displays the number of items remaining in the queue (items that have passed through the reflective sensor but have not been sorted) as well as a summary of all items that have already been sorted.

Ramp Down

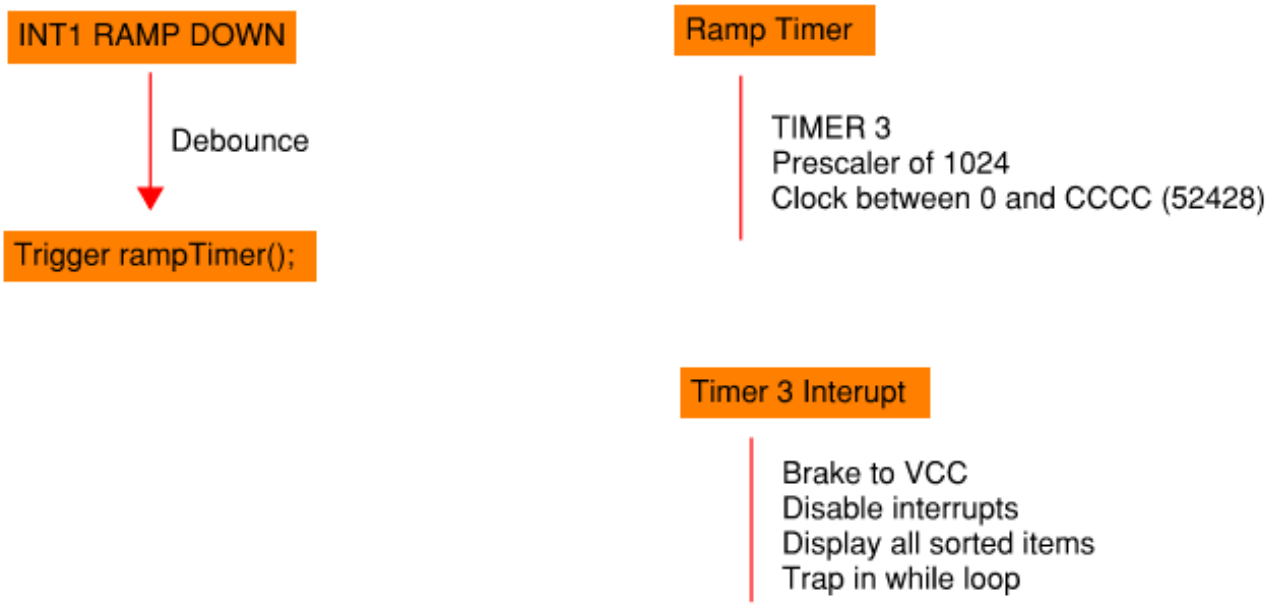


Figure 11: Ramp Down Interrupt Breakdown

The ramp-down interrupt, once debounced, initiates the rampTimer() function, which starts a timing sequence on Timer 3. The duration of the timer is calculated as follows:

$$Seconds\ per\ tick: \frac{1024\ pre\ scale}{8Mhz} = 0.000128s\ per\ tick$$

$$Total\ duration: 52428\ ticks * 0.000128s = 6.71\ seconds$$

Equation 1: Ramp Timer Duration

A separate timer was utilized to ensure it does not interrupt the functionality of the system, as one of the requirements involved being able to scan the objects that have not yet been added to the system and still sort them.

Once the timer finishes, all items added to the belt before hitting the button can be properly scanned and sorted. When the timer completes, a final interrupt is triggered. At this point, the belt motor is actively braked to VCC, all other interrupts are disabled, and a summary of the sorted objects is displayed. The program then enters an infinite while loop, effectively shutting the system down and preventing further user input or object processing until a manual reset occurs.

System Performance

Operating Parameters

The operating parameters of the sorting system were selected through iterative testing to achieve an optimal balance between speed, accuracy, and reliability. These values represent the conditions under which the system performs sorting tasks with minimal error.

The conveyor belt speed was set to operate at 50% of its maximum PWM duty cycle. This speed was chosen to reduce the likelihood of parts gaining excessive momentum, which could lead to inaccurate positioning or items being ejected from the sorting plate. Slower belt speeds also allowed the sensors to reliably capture data for each object without overlapping signals or missed detections.

For the sorting mechanism, an S-curve acceleration profile was used to control the stepper motor rotation [1]. This approach provided smooth acceleration and deceleration, minimizing vibration and mechanical shock that could occur with a linear acceleration profile. The smoother motion reduced the chance of objects being misaligned during the drop process and allowed the system to transition more quickly between sorting positions without overshooting or jerky motion. The following equation and parameters were utilized to form the 90 and 180 acceleration profiles shown in the following figures.

$$S(x) = \min + (max - \min) * \frac{1}{(1 + e^{-k(x-x_0)})^a}$$

Equation 2: S-Curve

Parameter	Value
Maximum Delay [ms]	15
Minimum Delay [ms]	5
x0	6
k	0.5
a	2

Table 1: S-Curve Parameters

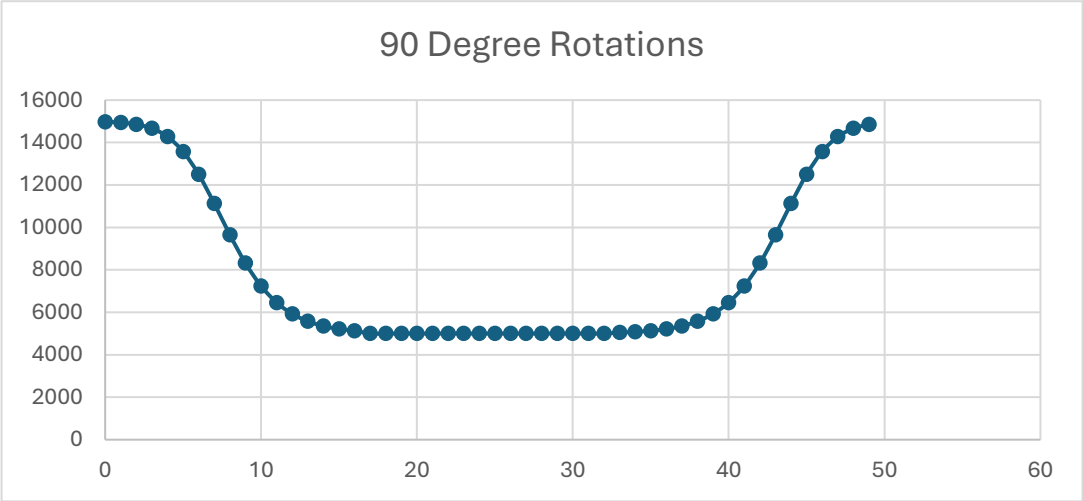


Figure 12: 90 Degree Rotation Acceleration Profile

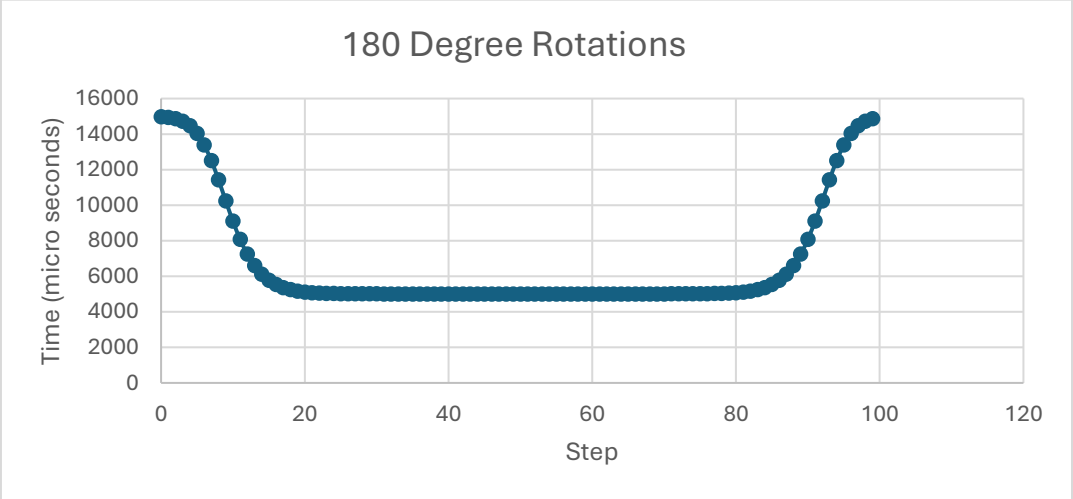


Figure 13: 180 Degree Rotation Acceleration Profile

To further enhance performance, the exit gate was designed with a nearly vertical angle. This decision minimized variability and eliminated the need for fine-tuning on the day of the demonstration, ensuring that each object dropped cleanly into its designated bin without bouncing or jamming. Test trials were conducted with the exit gate positioned at various angles. When angled toward the sorting plate, objects occasionally fell off the belt prematurely as the belt was braking to VCC. Angling the gate toward the loading side did not provide enough time for each object to reach the drop point before the belt stopped for the next item. Considering these outcomes, the optimal choice for the final demonstration was to keep the exit gate in a vertical position, ensuring consistent and reliable operation.

Numerous loading strategies were tested, beginning conservatively with four items per run. This configuration produced an error-free sort cycle of approximately 26 seconds. Following successful validation, the system was scaled to handle eight pieces per batch, reducing the total cycle time to 24 seconds with zero sorting errors, demonstrating both improved sorting time and system stability. During each run, our group aimed to minimize spacing between items while still maintaining a small gap to ensure the ADC could identify each piece individually. This proved effective, as the ADC successfully registered 12 readings for each object type, resulting in a total of 48 accurately identified pieces.

Electrical noise from the DC motor initially caused glitching at both the reflective sensor and exit gate sensor, leading to missed scans or extra interrupt triggers at the exit gate. To mitigate this, a combination of resistors and capacitors was added to each sensor input. These components filtered out voltage spikes and signal fluctuations, significantly improving sensor reliability. With noise suppression in place for the demonstration, the system was able to classify and sort objects consistently under varying load conditions.

Testing & Calibration Procedures

Calibrating Pieces

For the project, calibration and testing procedures were carefully implemented to ensure accurate material detection during the reflective sensing stage. Calibration was performed using black, white, steel, and aluminum pieces, with six samples of each type selected from the clean box set provided by the lab. The black pieces were deliberately handled to introduce skin oils, which improved reflectivity detection and sorting accuracy. In contrast, the white pieces were thoroughly cleaned with a microfiber cloth to reduce glare and inconsistencies caused by surface reflections. Each of the six pieces per material was passed through the reflective sensor, during which the ADC values were displayed on an LCD screen. The highest and lowest ADC readings for each material were recorded to establish threshold ranges for classification. It's important to note that this ADC output was only used during calibration. The output was disabled during actual testing to prevent processing delays. Displaying the ADC values in real-time on the LCD was found to slow down the system, potentially leading to misreads during sorting.

Aluminum Max	Steel Max	White Max	Black Max
100	550	925	1023

Table 2: ADC Maximum Readings

Calibrating Piece Drop Time & Position

The stepper motor position was calibrated to ensure precise alignment of the pieces on the sorting plate. On the day of the demo, it was observed that the pieces were not consistently landing in the center of the sorting plate after being released. To correct this, the stepper motor was manually adjusted by moving it one step counterclockwise immediately after the homing sequence. This small correction ensured each piece dropped accurately into the center, improving sorting accuracy. Additionally, an effective pre-drop system was implemented, where the piece is released 20 steps before the stepper finishes its full rotation to the target bucket. This timing allowed the piece to fall naturally and land exactly in the center of the sorting plate by the time the stepper reached its final position. To fine-tune this mechanism, slow-motion video recordings were used to closely observe the motion of the falling pieces. This helped verify that the pieces consistently avoided hitting the rim and landed cleanly in their respective buckets.

Stepper Motor Maximum Speed Testing

The stepper motor curve was calibrated by adjusting the previously mentioned equation through modifications to the maximum delay, minimum delay, and coefficient terms. To streamline the process, an Excel table was used where the stepper steps served as the input and the corresponding number of milliseconds (calculated using the curve function) served as the output. To efficiently transfer the updated timing values into the program, the Excel sheet employed the "TEXTJOIN" function, which concentrated the entire column of output values into a single cell. This allowed for quick copy-and-paste of the updated curve array directly into the code. This method provided a fast and effective way to fine-tune the stepper motor's performance through iterative adjustments.

Demonstration Results

The system's performance during evaluation was measured using the "System Performance Index" (SPI). The formula considers the amount of correctly sorted pieces, number of errors, and complete sort time to produce an SPI score. The SPI score is calculated as follows:

$$SPI = \frac{N_c - N_e}{T}$$

Equation 3: SPI Calculation

In this equation, N_c represents the number of correctly sorted pieces, N_e is the number of incorrectly sorted pieces, and T is the total time taken to complete the sorting process. As previously mentioned, our group achieved two successful zero error runs, one with a sort time of 24 seconds and the other with a sort time of 26 seconds.

Test Trial	N_c	N_e	T [s]	SPI
1	48	0	26	1.85
2	48	0	24	2.00

Table 3: Successful Test Trial SPI Scores

While faster speeds were technically achievable, they often resulted in trade-offs such as missed sensor reads, inaccurate object identification, or improper sorting. The selected parameters represent the recommended operating conditions for consistent and repeatable performance.

Limitations & Trade-offs

Throughout the design and testing process, several limitations and trade-offs were identified that impacted system performance and influenced design decisions. One of the most notable limitations was maintaining consistent spacing between objects on the conveyor belt. When items were placed too close together, the reflective sensor sometimes failed to trigger between objects. This prevented the ADC from registering a new minimum value, resulting in inaccurate or missed object readings. To address this, a minimum gap was used between items, which slightly reduced sorting time but significantly improved system reliability.

Another important trade-off involved balancing sorting speed with accuracy. While increasing the conveyor speed reduced total sort time, it also decreased the window for reliable sensor readings

and sorting plate alignment. After testing, a 50% PWM duty cycle speed was selected to ensure consistent performance while still maintaining a fast-paced sorting time.

The system also faced challenges related to stepper motor performance when weight was added to the sorting tray. Heavier loads introduced greater resistance, and if the s-curve acceleration profile was too aggressive, the stepper motor began to jitter during rotation. This caused positional errors that occasionally led to misaligned drops or failed sorting. To mitigate this, the s-curve profile was carefully tuned to provide a smoother ramp-up and ramp-down, reducing stress on the motor while maintaining acceptable speed. Additionally, the stepper delay was set to a minimum of 5 ms per step. Testing showed that any delay below this threshold created jittering in the motor, making 5 ms the optimal value for reliable and smooth rotation.

Additionally, hardware inconsistencies between lab stations introduced variability in performance. In particular, some stepper motors at certain stations performed poorly, with issues like missed steps or inconsistent rotation speeds when given our groups S-curve acceleration profile. These inconsistencies required additional setup time and tuning whenever the system was moved between stations, which affected repeatability and development efficiency.

Overall, these limitations required careful calibration and design adjustments to ensure the system remained accurate under various conditions.

Experience & Recommendations

It is highly recommended to start this project early, as many issues and glitches tend to arise during the final stage, especially during the 48-piece run. These problems are often difficult to predict and can take significant time to troubleshoot. Beginning the project ahead of schedule allows for more flexibility when resolving unexpected bugs and ensures there's enough time for multiple rounds of testing and refinement.

One specific issue encountered early on was related to belt stacking. When both the exit gate and the ADC were running simultaneously within the state machine, the ADC process could be interrupted by an exit gate trigger. This led to incorrect sorting behavior. A quick and effective solution was to move the enqueue operation inside the ADC system. This ensured that the ADC

completed its task fully before any interruptions could occur, resulting in more reliable piece sorting during operation.

Another useful tip is to schedule the demo run earlier in the morning. When the system is left running for long periods, stepper motor and driver tend to generate heat due to the running for long durations. Both the stepper and motor driver can overheat, and this thermal buildup often causes the stepper to miss steps causing mis sorts. Running the demo early reduces the chance of thermal issues since the system hasn't been in use for hours beforehand.

Lastly, writing clear and concise comments in the code significantly improves the debugging process. Not only does it help the original developer when returning to the code after some time, but it also benefits others, such as lab managers or classmates, who may need to assist in troubleshooting. Well-documented code is an invaluable tool when dealing with complex systems and should not be overlooked.

Conclusion

The automatic sorting system successfully met its design objectives, accurately identifying and sorting objects by material and color using a combination of sensors, motor control, and interrupt logic. Key features such as the s-curve acceleration for the stepper motor, a dynamic linked list queueing system, and tuned belt speed contributed to the system's performance. The system consistently sorted up to eight objects per batch with zero errors, demonstrating its accuracy and precision.

Throughout the project, several challenges were encountered, including sensor noise, motor jitter, and timing issues. These were addressed through hardware adaptations, software tuning, and iterative testing. Despite variations between lab stations and hardware limitations, the final design proved to be stable and effective. Overall, the project provided valuable experience in embedded system integration, control, and collaborative engineering problem-solving.

References

[1] Dr. K. Fordyce, “Some basics on the value of S curves and market adoption of a new product,” Arkieva, https://blog.arkieva.com/basics-on-s-curves/ (accessed Apr. 17, 2025).

[2] ATMEGA640/1280/1281/2560/2561 datasheet, https://ww1.microchip.com/downloads/en/devicedoc/atmel-2549-8-bit-avr-microcontroller-atmega640-1280-1281-2560-2561_datasheet.pdf (accessed Mar. 9, 2025).

[3] The University of Victoria, “MILESTONE 5: Final Project,” 2024.

Appendix

```
#include <avr/interrupt.h>
#include <avr/io.h>
#include <stdlib.h>
#include "lcd.h"
#include "LinkedQueue.h"
#include <stdbool.h>

// ********************Global Variables ********************
volatile char STATE;
volatile unsigned int dir = 2;
volatile unsigned int pause = 1;
volatile unsigned int curPos = 0;
volatile unsigned int ADC_result;
volatile unsigned int ADC_result_flag;
volatile unsigned int reflect_ADCL;
volatile unsigned int reflect_ADCH;
volatile unsigned int lowestScan = 1200;
volatile int pieceAtGate = 0;

// MATERIAL COUNTERS FOR PAUSE / RAMP DOWN
volatile unsigned int a = 0;
volatile unsigned int s = 0;
volatile unsigned int w = 0;
volatile unsigned int b = 0;

// DC & STEPPER MOTOR TABLES
const int dcTable[3] = { 0b00001111, 0b00001110, 0b00000111};  //0 = Brake VCC, 1 = clockwise,
2, counter clockwise
const int sTable[4] = { 0b00110101, 0b00110110, 0b00101110, 0b00101101 }; //output table for
stepper

// STEPPER ACCELERATION TABLES
const int stepperTable90[50] = {
      14994.2463645021, 14983.0620555507, 14952.5992453571, 14875.9737830886,
14702.0164230345, 14353.6800791064, 13750, 12854.1422931395, 11715.2744725744,
10460.8594363155, 9240.31451263476, 8166.74550655454, 7293.40360705343, 6623.02715016099,
6129.5538527468, 5776.87595243661, 5529.93937246223, 5000, 5000, 5000, 5000, 5000, 5000, 5000,
5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5242.88374608252, 5359.45792227191,
5529.93937246223, 5776.87595243661, 6129.5538527468, 6623.02715016099, 7293.40360705343,
8166.74550655454, 9240.31451263476, 10460.8594363155, 11715.2744725744, 12854.1422931395,
13750, 14353.6800791064, 14702.0164230345, 14875.9737830886, 14952.5992453571
};
```

```
const int stepperTable180[100] = {
      14994.2463645021, 14983.0620555507, 14952.5992453571, 14875.9737830886,
14702.0164230345, 14353.6800791064, 13750, 12854.1422931395, 11715.2744725744,
10460.8594363155, 9240.31451263476, 8166.74550655454, 7293.40360705343, 6623.02715016099,
6129.5538527468, 5776.87595243661, 5529.93937246223, 5359.45792227191, 5242.88374608252,
5163.68763406229, 5110.12048835884, 5073.99542991365, 5049.68152941845, 5033.33896266334,
5022.36417239656, 5014.99852842947, 5010.05713049983, 5006.74298694322, 5004.52062936766,
5003.0305696206, 5002.03158689488, 5001.36187423301, 5000.91291892491, 5000.6119601348,
5000.41021466321, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000,
5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 5000,
5000, 5000, 5000, 5000.18432410554, 5000.27497759113, 5000.41021466321, 5000.6119601348,
5000.91291892491, 5001.36187423301, 5002.03158689488, 5003.0305696206, 5004.52062936766,
5006.74298694322, 5010.05713049983, 5014.99852842947, 5022.36417239656, 5033.33896266334,
5049.68152941845, 5073.99542991365, 5110.12048835884, 5163.68763406229, 5242.88374608252,
5359.45792227191, 5529.93937246223, 5776.87595243661, 6129.5538527468, 6623.02715016099,
7293.40360705343, 8166.74550655454, 9240.31451263476, 10460.8594363155, 11715.2744725744,
12854.1422931395, 13750, 14353.6800791064, 14702.0164230345, 14875.9737830886,
14952.5992453571
};

// MATERIAL IDENTIFICATION
const int alum = 100;
const int steel = 550;
const int white = 925;
const int black = 1022;

//LINKED LIST SETUP
link *head;       // The ptr to the head of the queue
link *tail;       // The ptr to the tail of the queue
link *newLink;    // A ptr to a link aggregate data type (struct)
link *rtnLink;    // same as the above

// STEPPER ROTATION VARIABLES
int pos = 0;
int diff = 0;
int absDiff = 0;
int shortestRotation = 0;
int oppositeDiff = 0;

//FUNCTION DECLARTION
void mTimer(int count);
void rCW(int step);
void rCCW(int step);
void rampTimer();
void stepperTimer(int count);
```

```
int main(int argc, char *argv[]){
   //MTIMER SETUP
   CLKPR = 0x80;
   CLKPR = 0x01;     //  sets system clock to 8MHz
   TCCR1B |= _BV(CS11);
   TCCR4B |= _BV(CS41);

   //LCD SETUP
   InitLCD(LS_BLINK|LS_ULINE);
   LCDClear();

   rtnLink = NULL; // initialize newlink and returnlink
   newLink = NULL;
   setup(&head, &tail);

   STATE = 0;

   //PWM SETUP
   TCCR0A = _BV(WGM00) | _BV(WGM01);  //set WGM bits over TCCR0A and TCCR0B for fast mode
   TCCR0A |= _BV(COM0A1);    // COM0A1 (bit 7) = 1, COM0A0 (bit 6) = 0, leaves other bits
unchanged
   TCCR0B |= _BV(CS01) | _BV(CS00);     // sets CS01 bit (Table 16-9) and prescaler to 64
   OCR0A = 130; // set output compare register to 40% duty cycle (8 bit, 0 - 255)
   DDRB |= 0b11111111; // OC0A exists on bit 7 of PORTB, set this to output (Lecture 10
diagram)

   cli();      // Disables all interrupts

   DDRG = 0x00;      // Hall effect enable pin
   DDRC = 0xFF;      // just use as a display
   DDRA = 0xFF;      // stepper motor


   // Set up the Interrupt 0,3 options
   //External Interrupt Control Register A - EICRA (pg 110 and under the EXT_INT tab to the
right)
   // config the external interrupt ======================================
   EIMSK |= (_BV(INT2) | _BV(INT3) | _BV(INT1)| _BV(INT4)); // enable INT1 2 and 3
   EICRA |= (_BV(ISC21) | _BV(ISC20) | _BV(ISC11) | _BV(ISC10) | _BV(ISC31)); // INT1 INT2
rising edge, INT3 falling
   EICRB |= _BV(ISC41); //FALLING EDGE

   // config ADC ===========================================================
   // by default, the ADC input (analog input is set to be ADC0 / PORTF0
```

```
ADCSRA |= _BV(ADEN); // enable ADC
ADCSRA |= _BV(ADIE); // enable interrupt of ADC
ADMUX |= _BV(REFS0); // setup AVCC with AREF pin (ADLAR is right adjsuted)

// sets the Global Enable for all interrupts ========================
sei();

//Motor homing
int i = 0;
while((PING & 0b00000010) == 0b00000010){ // plate homing sequence
   PORTA = sTable[i];
   curPos = i; //record the last inputted position
   mTimer(15); //15ms delay for motor
   i++;
   if(i == 4) {
      i = 0;
   }
}



goto POLLING_STAGE;
// POLLING STATE
POLLING_STAGE:
switch(STATE){
   case (0) :
   goto POLLING_STAGE;
   break;
   case (1) :
   goto PAUSE;
   break;
   case (3) :
   goto BUCKET_STAGE;
   break;
   case (5) :
   goto END;
   default :
   goto POLLING_STAGE;
}//switch STATE


PAUSE:
//Display sorted pieces and remaining to be sorted (PS)
LCDClear();
LCDWriteString("A  S  W  B  PS");
```

```
   LCDGotoXY(0,1);
   LCDWriteInt(a, 2);
   LCDWriteString(" ");
   LCDWriteInt(s, 2);
   LCDWriteString(" ");
   LCDWriteInt(w, 2);
   LCDWriteString(" ");
   LCDWriteInt(b, 2);
   LCDWriteString(" ");
   LCDWriteInt(size(&head, &tail), 2);
   STATE = 0;
   goto POLLING_STAGE;

   BUCKET_STAGE:
   pieceAtGate = 0;
   if (head != NULL) {
      if (head->e.stage == pos) { //If on the correct position, drop the piece
         mTimer(30);
         PORTB = dcTable[2];
         } else {
         diff = head->e.stage - pos; // difference between current position and where the
plate needs to go
         absDiff = abs(diff); // determines how far plate needs to rotate
         oppositeDiff = 4 - absDiff; // alternative option
         if (absDiff <= oppositeDiff) {
            shortestRotation = absDiff; // if rotating directly is shorter
            } else {
            shortestRotation = oppositeDiff; // otherwise, go opposite direction
         }
         if ((absDiff <= oppositeDiff && diff > 0) || (absDiff > oppositeDiff && diff < 0)) {
// determines whether CW or CCW
            rCW(50 * shortestRotation); // either 90 or 180 degree turn
            } else {
            rCCW(50 * shortestRotation); // either 90 or 180 degree turn
         }
      }
      //Piece counter for pause / rampdown
      if(head ->e.stage == 1) {
         a++;
      }
      if(head ->e.stage == 3) {
         s++;
      }
      if(head ->e.stage == 2) {
         w++;
```

```
      }
      if(head ->e.stage == 0) {
         b++;
      }
      pos = head->e.stage; // set current position to to moved one
      dequeue(&head, &rtnLink, &tail); // dequeue from FIFO
      free(rtnLink); //free memory
   }
   if(pieceAtGate == 1){
      STATE = 3; // if another piece is currently at exit gate after predrop
   }else{
      STATE = 0; // if no piece at gate
   }
   goto POLLING_STAGE;

   END:
   return(0);
}

ISR(INT1_vect){
   mTimer(5); //debounce
   while ((PIND & 0b00000010) == 0b00000010) {}
   LCDClear();
   LCDWriteString("Ramping down...");
   rampTimer(); //Start 6.7s ramp down timer
}

ISR(INT2_vect){
   //Optical sensor
   lowestScan = 1023; //Reset to highest possible value as system is looking for lowest
scanned
   ADCSRA |= _BV(ADSC); //Start ADC
}

ISR(INT3_vect){
   if((PIND & 0b00001000) == 0b00000000){ //Debounce
      mTimer(20);
      if(pause){
         LCDClear();
         dir = 2; //set direction to CW
         pause = 0; //if paused, unpause
      }
      else{
         dir = 0; // brake to VCC
         pause = 1; //enable pause
```

```
         STATE = 1;
      }
      PORTB = dcTable[dir];
      while((PIND & 0b00001000) == 0b00000000){} // debounce
      mTimer(20);
   }
}

ISR(INT4_vect){
   PORTB = dcTable[0]; //brake to VCC
   pieceAtGate = 1; //set flag
   STATE = 3;
}

ISR(ADC_vect)
{
   reflect_ADCL = ADCL; //8 bit
   reflect_ADCH = ADCH; //remaining 2 bits
   ADC_result = reflect_ADCL; //combine bits
   ADC_result |= (reflect_ADCH << 8); //combine bits
   if(ADC_result < lowestScan) { //Until lowest found, keep setting lowest scan to ADC value
      lowestScan = ADC_result;
   }
   if ((PIND & 0b00000100) == 0b00000100)
   {
      ADCSRA |= _BV(ADSC);
      } else {
      initLink(&newLink);
      newLink ->e.itemCode = lowestScan;
      //LCDClear();                        //used for calibration of pieces, remove when
running
      //LCDWriteInt(lowestScan, 4);        //used for calibration of pieces, remove when
running
      if(lowestScan < alum) { //Piece stage identifier
         newLink -> e.stage = 1;
      }
      if (lowestScan > alum && lowestScan < steel) {
         newLink -> e.stage = 3;
      }
      if (lowestScan > steel && lowestScan < white) {
         newLink -> e.stage = 2;
      }
      if (lowestScan > white && lowestScan < black) {
         newLink -> e.stage = 0;
      }
```

```
      enqueue(&head, &tail, &newLink);
   }
}

ISR(TIMER3_COMPA_vect){
   PORTB = dcTable[0]; // brake to VCC
   cli(); // disable interrupts
   LCDClear();
   LCDWriteString("A  S  W  B");
   LCDGotoXY(0,1);
   LCDWriteInt(a, 2);
   LCDWriteString(" ");
   LCDWriteInt(s, 2);
   LCDWriteString(" ");
   LCDWriteInt(w, 2);
   LCDWriteString(" ");
   LCDWriteInt(b, 2);
   while(1){} // program must be reset to be used again
}

// If an unexpected interrupt occurs (interrupt is enabled and no handler is installed,
// which usually indicates a bug), then the default action is to reset the device by jumping
// to the reset vector. You can override this by supplying a function named BADISR_vect which
// should be defined with ISR() as such. (The name BADISR_vect is actually an alias for
__vector_default.
// The latter must be used inside assembly code in case <avr/interrupt.h> is not included.
ISR(BADISR_vect)
{
   LCDClear();
   LCDWriteString("BAD ISR");
   while (1) {} //user must reset system if a BAD ISR is tripped
}

//Given in previous lab
void mTimer(int count){
   int i;
   i = 0;
   TCCR1B |= _BV(WGM12);
   OCR1A = 0x03E8;
   TCNT1 = 0x0000;
   TIFR1 |= _BV(OCF1A);
   while(i<count){
      if((TIFR1 & 0x02) == 0x02){
         TIFR1|= _BV(OCF1A);
         i++;
```

```
      }
   }
   return;
}

// ********** STEPPER CONTROLS
void rCW(int step) {
   int i = 0;
   while(i < step) {
      if(curPos == 3) { //check if out of table bounds
         curPos = 0;
         } else {
         curPos++;
      }
      PORTA = sTable[curPos]; //move stepper through table
      if(step == 50) { //90 deg rotation
         stepperTimer(stepperTable90[i]); //S curve 90deg
         if(i == 30){ //predrop by 20 steps
            PORTB = dcTable[2];
         }
      }
      if (step == 100) { //180 deg rotation
         stepperTimer(stepperTable180[i]); //S curve 180deg
         if(i == 80){ //predrop by 20 steps
            PORTB = dcTable[2];
         }
      }
      i++;
   }
}

void rCCW(int step) {
   int i = 0;
   while(i < step) {
      if(curPos == 0) { //check if out of table bounds
         curPos = 3;
         } else {
         curPos--; //continue as normal
      }
      PORTA = sTable[curPos]; //move through stepper
      if(step == 50) {
         stepperTimer(stepperTable90[i]); //S Curve 90deg
         if(i == 30){ //pre drop by 20 steps
            PORTB = dcTable[2];
         }
```

```
      }
      if (step == 100) {
         stepperTimer(stepperTable180[i]); //S curve 180deg
         if(i == 80){ //pre drop by 20 steps
            PORTB = dcTable[2];
         }
      }

      i++;
   }
}

//Ramp down
void rampTimer() {
   TCCR3B |= _BV(WGM32); // clear timer on compare
   TCCR3B |= _BV(CS32) | _BV(CS30); // Prescaler of 1024
   OCR3A = 0xCCCC; //sets output compare register to about 52000 cycles ~ 6.7s
   TCNT3 = 0x0000; // sets initial value for timer compare to 0
   TIMSK3 = _BV(OCIE3A); //enable the output compare interrupt enable
   TIFR3 |= _BV(OCF3A); //clear the timer interrupt flag
}

void stepperTimer(int count){
   TCCR4B |= _BV(WGM42); // CTC mode
   OCR4A = count; // set output compare register to count
   TCNT4 = 0x0000;//set initial value of timer to 0
   TIFR4 |= _BV(OCF4A);// clear interrupt
   while((TIFR4 & 0x02) != 0x02){} // wait for OCR4A flag to be set
}

// ********** LINKED LIST FUNCTIONS
void setup(link **h,link **t){
   *h = NULL;     /* Point the head to NOTHING (NULL) */
   *t = NULL;     /* Point the tail to NOTHING (NULL) */
   return;
}/*setup*/




/*****************************************************************************
* DESC: This initializes a link and returns the pointer to the new link or NULL if error
* INPUT: the head and tail pointers by reference
*/
void initLink(link **newLink){
```

```
   //link *l; //Error found in link list provided
   *newLink = malloc(sizeof(link));
   (*newLink)->next = NULL;
   return;
}/*initLink*/




/*****************************************************************************
*  DESC: Accepts as input a new link by reference, and assigns the head and tail
*  of the queue accordingly
*  INPUT: the head and tail pointers, and a pointer to the new link that was created
*/
/* will put an item at the tail of the queue */
void enqueue(link **h, link **t, link **nL){
   if (*t != NULL){
      /* Not an empty queue */
      (*t)->next = *nL;
      *t = *nL; //(*t)->next;
      }/*if*/
      else{
         /* It's an empty Queue */
         //(*h)->next = *nL;
         //should be this
         *h = *nL;
         *t = *nL;
         }/* else */
         return;
}/*enqueue*/




/*****************************************************************************
* DESC : Removes the link from the head of the list and assigns it to deQueuedLink
* INPUT: The head and tail pointers, and a ptr 'deQueuedLink'
*      which the removed link will be assigned to
*/
/* This will remove the link and element within the link from the head of the queue */
void dequeue(link **h, link **deQueuedLink, link **t){
   /* ENTER YOUR CODE HERE */
   *deQueuedLink = *h;  // Will set to NULL if Head points to NULL
   /* Ensure it is not an empty queue */
   if (*h != NULL){
```

```
      *h = (*h)->next;
   }
   if((*h == NULL)){ // if list is empty after dequeue (red herring in initial code)
      *t = NULL; // point tail to null
      }/*if*/
      return;
      }/*dequeue*/




/*****************************************************************************
* DESC: Peeks at the first element in the list
* INPUT: The head pointer
* RETURNS: The element contained within the queue
*/
/* This simply allows you to peek at the head element of the queue and returns a NULL pointer
if empty */
element firstValue(link **h){
   return((*h)->e);
   }/*firstValue*/




/*****************************************************************************
* DESC: deallocates (frees) all the memory consumed by the Queue
* INPUT: the pointers to the head and the tail
*/
/* This clears the queue */
void clearQueue(link **h, link **t){

   link *temp;

   while (*h != NULL){
      temp = *h;
      *h=(*h)->next;
      free(temp);
      }/*while*/

      /* Last but not least set the tail to NULL */
      *t = NULL;

      return;
```

```
      }/*clearQueue*/




/*****************************************************************************
* DESC: Checks to see whether the queue is empty or not
* INPUT: The head pointer
* RETURNS: 1:if the queue is empty, and 0:if the queue is NOT empty
*/
/* Check to see if the queue is empty */
char isEmpty(link **h){
   return(*h == NULL);
   }/*isEmpty*/




/*****************************************************************************
* DESC: Obtains the number of links in the queue
* INPUT: The head and tail pointer
* RETURNS: An integer with the number of links in the queue
*/
/* returns the size of the queue*/
int size(link **h, link **t){

   link  *temp;         /* will store the link while traversing the queue */
   int   numElements;

   numElements = 0;

   temp = *h;        /* point to the first item in the list */

   while(temp != NULL){
      numElements++;
      temp = temp->next;
      }/*while*/

      return(numElements);
      }/*size*/
```